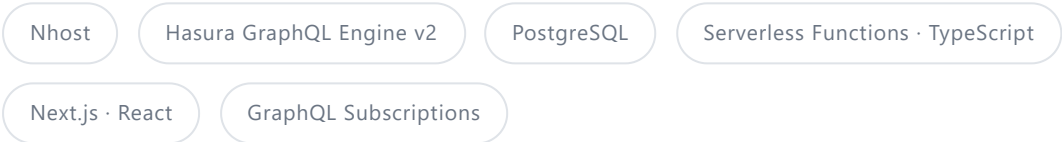


AI Agent Workflow Builder

A multi-tenant, low-code platform for composing and executing agentic workflows — with permission enforcement split across a declarative database layer and a procedural handler layer.



DOCUMENT

Architecture & Design
Rationale

SCOPE

Data model · Permissions · Execution
engine

SCHEMA

17 tables · 1 view · 4 Actions · 2 Event
Triggers

01 System Overview

What the platform does, and how the moving parts divide responsibility.

The Workflow Builder lets users inside an organization compose workflows from typed steps — LLM calls, HTTP requests, database writes, notifications, conditional branches, and approval gates — then start them manually, by webhook, on a schedule, or from a database change.

Every workflow belongs to exactly one organization. A user's rights are never global: they are resolved per-organization through the `org_members` junction table, so the same person can be an owner in one org and a viewer in another. That constraint shapes both the schema and the permission model.

Component responsibilities

COMPONENT	RUNS AS	RESPONSIBILITY
PostgreSQL	–	Source of truth. Enforces tenancy invariants with triggers and constraints that the application cannot bypass.
Hasura	<code>role: user</code>	Auto-generated CRUD and live subscriptions. Applies declarative row-level filters — Permission Layer 1.
Action handlers	<code>admin secret</code>	Business rules that a static row filter cannot express: role re-derivation, quota, run orchestration — Permission Layer 2.
Event handlers	<code>admin secret</code>	React to row changes: start runs from watched tables, deliver queued notifications.
Next.js client	JWT	Workflow editor and live run viewer. Holds no authority — every claim it makes is re-checked server-side.

THE LOAD-BEARING IDEA

Action handlers execute with the admin secret, which means Hasura's row permissions *do not run* for them. Layer 1 therefore cannot protect Layer 2 — so every handler independently re-derives the caller's role from `org_members` before acting. The client's `x-hasura-role` is treated as a claim, never as an authorization.

Request paths

Read & edit

`client` → `Hasura` → `Postgres`

Workflow definitions, steps, triggers, and run history.
Guarded entirely by declarative row filters. No handler is involved.

Execute

`client` → `Action` → `runner` → `Postgres`

Starting, approving, or rejecting a run. Guarded by procedural checks inside the handler, because the decision depends on live state.

Observe

`client` → Hasura subscription

A subscription on `step_runs` filtered to one `workflow_run_id` streams progress live, including the paused-awaiting-approval state.

React

Postgres → Event Trigger → handler

A row inserted into a watched table fires an Event Trigger, which starts a run without any user in the loop.

02 Data Model

Seventeen tables, but only four ideas. Each layer below owns one of them; a column-level reference follows in the appendix.

Schema map



AGGREGATION

The view `org_usage_current_month` exposes quota limit, used, remaining, runs this month, failed runs, and average successful run duration — computed in Postgres and tracked by Hasura, so the client never aggregates client-side over rows it would otherwise need permission to read.

03 Why the Schema Looks Like This

Six decisions that each close a specific failure mode.

01 Every tenant-owned row carries its own `org_id`

Steps, triggers, runs, step runs, and artifacts all store `org_id` directly rather than reaching it through a foreign-key chain. This keeps the permission filter a single join to `org_members` instead of a three-table walk, and it keeps the index `(user_id, org_id, role)` covering for every check.

02 ...but the client never gets to set it

Denormalization is only safe if the copy cannot lie. A `BEFORE INSERT` trigger on each child table overwrites `org_id` with the value read from its parent. A caller who smuggles a foreign `org_id` into a mutation has it silently replaced before the row lands — so the permission filter and the stored value can never disagree. The triggers deliberately do *not* fire on `UPDATE : org_id` is immutable once written, and firing on update broke cascading deletes.

03 Step runs snapshot the step, rather than pointing at it

`step_runs` copies `name`, `type`, and `config` at execution time. Editing a workflow while it is mid-flight therefore cannot redirect a running execution, and historical runs keep describing what actually ran rather than what the definition says today. The `step_id` link is kept for traceability but is `SET NULL` on delete.

04 Execution state is unwritable by users

`workflow_runs` and `step_runs` grant the `user` role *select only* — no insert, no update, no delete. Only the admin-secret handlers mutate them. This removes an entire class of escalation where a viewer sets a status directly instead of passing through the approval path.

05 Enums are tables, not native PG types

Hasura generates real GraphQL enum types from lookup tables and enforces referential integrity against them, while adding a new step type stays a plain `INSERT` — no migration, no type rewrite, no downtime.

06 Constraints encode the invariants that matter

- `UNIQUE (workflow_id, position)` — DEFERRABLE, so a drag-to-reorder can shuffle positions inside one transaction.
- `UNIQUE (workflow_run_id, position)` on step runs — makes a duplicated Event Trigger retry a no-op instead of a double execution.
- A deferred constraint trigger verifies both branch targets belong to the *same* workflow, so a conditional branch can never jump execution into another org's workflow.
- `CHECK (max_attempts BETWEEN 1 AND 10)` — bounds retry cost.

04 The Two Permission Layers

Layer 1 answers “may you see or touch this row?” declaratively. Layer 2 answers “may you do this now, given live state?” procedurally.

Layer 1 — declarative row-level security

Only two roles reach the GraphQL API: `public` and `user`. There is no privileged Hasura role, because privilege is not a property of the session — it is a property of the (user, org) pair. Every filter therefore resolves through `org_members`:

```
// workflows.select - reachable only through your own membership
{ org: { members: { user_id: { _eq: X-Hasura-User-Id } } } }

// workflow_steps.insert - org scope AND step-type gate, combined
{ _and: [
  { workflow: { org: { members: {
    user_id: { _eq: X-Hasura-User-Id },
    role: { _in: ["owner", "editor"] } } } } },
  { _or: [
    { type: { _nin: ["db_write", "notify"] } },
    { workflow: { org: { members: {
      user_id: { _eq: X-Hasura-User-Id },
      role: { _eq: "owner" } } } } } ] } ] }
```

Because the join is re-evaluated on every request, a user who guesses a valid ID from another organization gets an *empty result*, not an error — the filter reveals nothing about whether the row exists.

EFFECTIVE MATRIX

OPERATION	OWNER	EDITOR	VIEWER	ENFORCED BY
Read anything in own org	yes	yes	yes	Hasura filter
Create / edit workflows & steps	yes	yes	no	Hasura filter
Add a <code>db_write</code> or <code>notify</code> step	yes	no	no	Hasura <code>_or</code> gate
Add a <code>webhook</code> trigger	yes	no	no	Hasura <code>_or</code> gate
Manage org membership	yes	no	no	Hasura filter
Trigger a run	yes	yes	no	Handler
Approve / reject a gate	yes	yes	no	Handler + state check
Write run or step-run state	no	no	no	No permission granted

WHY THE STEP-TYPE GATE LIVES IN HASURA TOO

Steps that reach outside the sandbox — `db_write` and `notify` — and inbound `webhook` triggers are restricted to owners at the row-permission level, not just in the UI. The gate is expressible as a static predicate over the row being written, so it belongs in Layer 1 where it cannot be skipped by calling the API directly.

Layer 2 — procedural guards in the handlers

Approving an `approval_gate` is not a row read or a row write — it is a mid-execution decision that depends on the run's *current* state. No static filter can express it, so it lives in code. Every handler follows the same four-move opening:

- 1 Authenticate the caller of the endpoint.** Nhost functions are publicly routable, so each handler verifies the `nhost-webhook-secret` header first. Without it, anyone could POST a forged Action payload and drive the engine directly.
- 2 Take the user ID from the verified JWT only.** Read from `session_variables['x-hasura-user-id']`, which Hasura populates after validating the token — never from the request body.
- 3 Re-derive the role from the database.** Query `org_members` for the (user, org) pair and compare it against the minimum rank the operation requires. The role the client claims is ignored entirely.
- 4 Fail identically for “not a member” and “wrong role.”** Both return the same `403 forbidden`, so a caller cannot probe which organizations exist by comparing responses. A missing workflow is the only `404`.

THE FOUR ACTIONS

ACTION	MIN. ROLE	ADDITIONAL GATE
<code>triggerWorkflowRun</code>	editor	Org quota must have a free unit — reserved atomically <i>before</i> the run row is created.
<code>approveStep</code>	editor	Target must actually be an <code>approval_gate</code> currently in <code>awaiting_approval</code> ; anything else is a 409.
<code>rejectStep</code>	editor	Same state check; fails the run instead of resuming it.
<code>webhookTriggerRun</code>	—	Authenticated by an unguessable per-trigger <code>webhook_token</code> rather than a user session; the trigger row supplies the org.

QUOTA IS RESERVED, NOT COUNTED

A read-then-write quota check races: two concurrent triggers both read `used < limit` and both proceed. Instead the engine issues a single conditional `UPDATE ... WHERE quota_used < limit` and inspects `affected_rows`. The unit is reserved before any run row exists, and released — clamped so it can never go negative — if the run fails.

05 Execution Engine

One runner serves all four trigger paths, and the same entry point both starts a run and resumes a paused one.

How a run starts

TRIGGER	ENTRY POINT	AUTHENTICATED BY
Manual	Action · triggerWorkflowRun	User JWT, re-checked against <code>org_members</code>
Webhook	Action · webhookTriggerRun	Unguessable per-trigger token (unique-constrained)
Scheduled	Cron → same runner	Platform-internal; trigger row supplies the org
Database event	Event Trigger · lead-created	Webhook secret header from Hasura

All four converge on `executeWorkflowRun(runId)`. Whatever started the run, the orchestration below is identical.

Run state machine



Step runs carry their own parallel status — `pending` `running` `awaiting_approval` `succeeded` `failed` `skipped` — and it is this table the client subscribes to for live progress.

The step loop

- 1 Load the run, its ordered step definitions, and any `step_runs` that already exist. Rebuild the templating context from already-succeeded steps, so a resumed run can still reference earlier outputs without re-running them.
- 2 For each step in order, **skip anything already in a final state**. This is what makes a duplicated Event Trigger delivery a no-op rather than a double execution — reinforced by the unique index on `(workflow_run_id, position)`.
- 3 Render the step's JSONB config against the run context, so a prompt or URL can interpolate the output of any earlier step.
- 4 Execute by type. `llm_call` and `http_request` make real external calls with bounded retry; `db_write` writes an artifact; `notify` queues a notification row; `conditional_branch` evaluates a predicate and returns the next step ID to jump to.
- 5 On success, persist the output and fold it into the context for later steps. On failure, mark the step `failed`, fail the run, and release the reserved quota unit.

- 6 On reaching an `approval_gate`, insert the step run as `awaiting_approval`, set the run to `paused`, and **return** — the handler exits rather than blocking.

RETRY POLICY

Retries use exponential backoff, but only for errors worth retrying: `5xx`, `429`, and `408` are transient; any other `4xx` is a permanent failure and is thrown immediately. Retrying a `401` just burns the function's execution budget and delays the error the user needs to see.

06 Approval Gates — Pause and Resume

The one place where the two permission layers must cooperate, and the reason Layer 2 has to exist at all.

An approval gate is an ordinary step. What makes it different is that the runner stops at it and returns, leaving the run durably suspended in the database rather than held open in a function's memory. Nothing is waiting; there is no timeout to exceed and no process to lose.

Pause

- 1 The runner reaches a step of type `approval_gate` and inserts its `step_run` directly as `awaiting_approval`.
- 2 The parent run moves to `paused`.
- 3 The handler returns the paused step-run ID to the caller and exits. Because the state lives in Postgres, the subscription on `step_runs` surfaces the pause to every watching client immediately.

Resume

- 1 A user calls `approveStep(step_run_id)`.
- 2 The approver's role is re-derived from `org_members` for the run's org and must be owner or editor. This is the check that cannot be a database permission: the handler runs as admin, so nothing else would enforce it.
- 3 The step's state is re-validated. It must genuinely be an `approval_gate` and still be in `awaiting_approval`. Approving a step that already completed, or a step of any other type, returns `409` — this is what makes a double-submit or a replayed request harmless.
- 4 The step run is stamped `succeeded` with `approved_by` and `approved_at`, and the parent run returns to `running`.
- 5 The runner is re-entered with `resumeAfterPosition` set to the gate's position, so it continues at the next step — with the context rebuilt from the steps that already succeeded — until the run completes or hits another gate.

WHY THE STATE CHECK MATTERS AS MUCH AS THE ROLE CHECK

Role alone is not sufficient. Without the state check, a legitimate editor could re-approve a gate that had already been passed, re-entering the runner and re-executing everything after it. The gate is authorized *and* single-use, and both properties are enforced in the same handler.

`rejectStep` is the mirror image: the same role and state checks, but it marks the step `failed` with the rejector's note and fails the run instead of resuming it.

Notification delivery follows the same shape. A `notify` step does not call Slack or an email provider inline — it inserts a row into `notifications`. An Event Trigger on that insert delivers it out of band with its own retry policy, so a flaky provider slows delivery rather than failing the workflow run.

A Appendix — Column Reference

Columns that carry meaning. Audit fields (`id`, `created_at`, `updated_at`) are present on every table and omitted here.

TENANCY

TABLE	COLUMNS	NOTES
organizations	name, slug, quota_limit, quota_used, quota_period_start	<code>slug</code> unique; a CHECK keeps both quota counters non-negative. The period rolls forward on first use in a new month.
org_members	org_id, user_id, role	Unique on (<code>org_id</code> , <code>user_id</code>) . Indexed (<code>user_id</code> , <code>org_id</code> , <code>role</code>) — kept covering because it drives every permission check.

DEFINITION

TABLE	COLUMNS	NOTES
workflows	org_id, name, description, is_active, created_by	<code>created_by</code> is SET NULL on user delete, so history survives an offboarding.
workflow_steps	workflow_id, org_id, position, name, type, config, max_attempts, on_true_step_id, on_false_step_id	<code>position</code> unique per workflow but DEFERRABLE. Branch targets validated against the same workflow by a deferred constraint trigger.
workflow_triggers	workflow_id, org_id, type, is_enabled, config, webhook_token, cron_expr, next_run_at	<code>webhook_token</code> globally unique — it is the credential for the inbound endpoint. Indexed on (<code>type</code> , <code>is_enabled</code> , <code>next_run_at</code>) so the scheduler can find due rows cheaply.

EXECUTION

TABLE	COLUMNS	NOTES
workflow_runs	workflow_id, org_id, trigger_id, trigger_type, triggered_by, status, input, error, started_at, finished_at	<code>trigger_id</code> is nullable and SET NULL: deleting a trigger must not erase the runs it produced. Indexed for both “latest runs of a workflow” and “open runs in an org.”
step_runs	workflow_run_id, step_id, org_id, position, name, type, config, status, input, output, error, attempt, max_attempts, approved_by, approved_at, approval_note, started_at, finished_at	<code>name</code> / <code>type</code> / <code>config</code> are snapshots, not references. Unique on (<code>workflow_run_id</code> , <code>position</code>) for idempotency.

SIDE EFFECTS

TABLE	COLUMNS	NOTES
workflow_artifacts	org_id, workflow_run_id, step_run_id, key, value	The sole target of <code>db_write</code> . A step never issues arbitrary SQL.
notifications	org_id, workflow_run_id, step_run_id, channel, target, subject, body, status, error, sent_at	Outbox pattern — insert queues, an Event Trigger delivers and stamps the result.
org_usage_events	org_id, workflow_run_id, kind, units	Ledger behind the quota counter, indexed (<code>org_id, created_at DESC</code>).
leads	org_id, email, name, note	Watched table demonstrating the <code>database_event</code> trigger path.

DATABASE ROUTINES

ROUTINE	PURPOSE
set_updated_at()	Shared BEFORE UPDATE timestamp trigger.
set_org_id_from_workflow()	Derives <code>org_id</code> on steps, triggers, and runs from the parent workflow. INSERT only.
set_org_id_from_run()	Same, for step runs and artifacts, from the parent run.
check_branch_targets()	Deferred constraint trigger — both branch targets must live in the same workflow.
consume_org_quota(org, units)	Rolls the quota period forward if the month changed, then reserves atomically. Returns <code>false</code> if the limit would be exceeded.
release_org_quota(org, units)	Returns a reservation, clamped at zero.
org_usage_current_month	View — the org-level aggregation exposed through Hasura.